

Cold Room Temperature Monitoring Project Documentation

Generated: 2026-06-04

Repository: COLD_ROOM_PROJECT

Remote: https://github.com/projectspacificcontrol/COLD_ROOM_PROJECT.git

Latest reviewed commit: 318972c add proprietary license

License: Proprietary, all rights reserved.

1. Executive Summary

The Cold Room Temperature Monitoring project is a production-oriented full-stack monitoring system for industrial cold rooms. It is designed to display live temperature readings for 18 cold rooms organized into 6 groups, with 3 rooms per group and 5 expected temperature sensors per room. The expected total is 90 sensors.

The system includes:

- ? React, TypeScript, Vite, and Tailwind frontend dashboard.
- ? FastAPI backend with normalized ingestion, room/sensor mapping, admin APIs, report export, health endpoints, and live updates.
- ? SQL Server production database model with SQLite local fallback.
- ? Secure admin panel for users, thresholds, IP allowlist, reports, alerts, system health, audit logs, and settings.
- ? Docker Compose deployment files for local and production use.
- ? Nginx reverse proxy configuration for local and HTTPS production deployments.
- ? Alembic migrations and seed command.
- ? Backend, frontend, and Playwright tests.
- ? CI workflow for GitHub Actions.
- ? Proprietary license restricting unauthorized copying and use.

This document explains the repository, runtime architecture, setup steps, operating model, security controls, permission flow, data flow, reports, deployment, troubleshooting, maintenance, and audit findings.

2. Repository Structure

```
COLD_ROOM_PROJECT/  
  .github/  
    workflows/  
      ci.yml  
  apps/  
    web/  
      Dockerfile  
      Dockerfile.prod  
      nginx.default.conf  
      package.json  
      playwright.config.ts
```

```

src/
  App.tsx
  admin/
  api/
  components/
  config/
  hooks/
  types/
tests/
  e2e/
  unit/
docs/
  admin-user-guide.md
  api-contract.md
  architecture.md
  backup-restore.md
  client-handover.md
  data-model.md
  database-schema.md
  deployment.md
  handover.md
  security.md
  security-checklist.md
  troubleshooting.md
  PROJECT_DOCUMENTATION.md
infra/
  docker-compose.yml
  docker-compose.prod.yml
  nginx/
    nginx.conf
    nginx.prod.conf
scripts/
  dev.ps1
  stop-dev.ps1
services/
  api/
    Dockerfile
    Dockerfile.prod
    alembic.ini
    alembic/
  app/
    api/
    core/
    db/
    models/
    schemas/
    scripts/
    services/
  tests/
.env.example
.gitignore
DELIVERY_REPORT.md
LICENSE
README.md

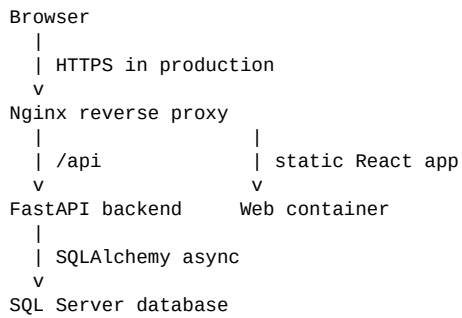
```

2.1 Important Root Files

- ? `.env.example`: Safe environment variable template. It contains placeholders only and must be copied to `.env` locally.
- ? `.gitignore`: Excludes `.env`, local DB files, logs, `node_modules`, build output, test reports, certificates, backups, and other non-Git artifacts.
- ? `LICENSE`: Proprietary license. This is not MIT. It restricts copying, modification, hosting, distribution, and reuse without written permission.
- ? `README.md`: Main project setup and navigation guide.
- ? `DELIVERY_REPORT.md`: Delivery-readiness summary and latest QA command results.

3. System Architecture

The system is split by deployable boundary.



3.1 Frontend

Location: apps/web

Technology:

- ? React
- ? TypeScript
- ? Vite
- ? Tailwind CSS
- ? Recharts
- ? Zod
- ? Lucide icons
- ? Playwright for e2e tests
- ? Vitest and Testing Library for unit/component tests

The frontend contains:

- ? Main cold room dashboard.
- ? Admin login and admin panel.
- ? Reusable dashboard components.
- ? Runtime response validation with Zod.
- ? SSE live update client with polling fallback.
- ? Fixture provider for development/demo mode.

3.2 Backend

Location: services/api

Technology:

- ? Python FastAPI
- ? SQLAlchemy async ORM
- ? Alembic migrations
- ? bcrypt password hashing
- ? python-jose JWT session token
- ? openpyxl Excel parsing/export

? aiiodbc/pyodbc SQL Server support

? aiosqlite local fallback

The backend contains:

? Dashboard APIs.

? Admin APIs.

? Authentication and current-user APIs.

? IP allowlist middleware.

? Security headers middleware.

? In-memory rate limiting middleware.

? Excel/API-shaped ingestion pipeline.

? Room/sensor normalizer.

? Threshold evaluation.

? Alert and sensor fault creation.

? Excel report export.

? Health, readiness, and metrics endpoints.

3.3 Database

Production database target: Microsoft SQL Server.

Local fallback: SQLite for development when no SQL Server URL is available.

The application uses Alembic migrations and SQLAlchemy models. Production should use SQL Server through:

```
DATABASE_URL=mssql+aiiodbc://user:password@host:1433/  
cold_room?driver=ODBC+Driver+18+for+SQL+Server&TrustServerCertificate=yes
```

4. Room And Sensor Model

The system expects 6 cold room groups.

Group	Rooms
CR123	1, 2, 3
CR456	4, 5, 6
CR789	7, 8, 9
CR101112	10, 11, 12
CR131415	13, 14, 15
CR161718	16, 17, 18

Each group has 15 source sensors:

Source Sensor Range	Room In Group	Dashboard Labels
TT01 to TT05	first room	T1 to T5
TT06 to TT10	second room	T1 to T5
TT11 to TT15	third room	T1 to T5

Expected totals:

- ? 18 rooms.
- ? 90 expected sensors.
- ? 5 displayed sensors per room.

The mapping is defined in `services/api/app/core/room_config.py`.

5. Data Ingestion And Normalization

The backend accepts wide source rows shaped like the uploaded Excel sample:

```
LogTime, CR123_TT01, CR123_TT02, ..., CR161718_TT15
```

The ingestion path is:

```
Excel file or live API row
-> provider/parser
-> normalizer
-> threshold/status evaluator
-> raw_ingest_logs
-> temperature_readings
-> alerts/sensor_faults
-> current_room_snapshots
-> dashboard APIs and live stream
```

Important behavior:

- ? LogTime is required and converted to UTC.
- ? Every source row is expanded into the 90 expected sensor slots.
- ? If an expected sensor column is missing, the system stores it as unavailable/faulted.
- ? Invalid values become invalid/faulted.
- ? Impossible values become impossible/faulted.
- ? Stale records can become offline depending on threshold stale timeout.
- ? Duplicate raw records are deduplicated by source plus LogTime.

6. Frontend Dashboard

Main file: `apps/web/src/App.tsx`

Dashboard components:

- ? `OverviewCards.tsx`: Total rooms, sensors, average temperature, active alerts, sensor faults.
- ? `FilterTabs.tsx`: All, normal, warning, critical, fault/offline filters.
- ? `RoomGroup.tsx`: Group section.
- ? `RoomCard.tsx`: Room card with average temperature, status badge, and T1-T5 chips.
- ? `RoomDrawer.tsx`: Side drawer for full room details, sensors, history chart, fault state, and alerts.
- ? `useDashboard.ts`: Dashboard state, selected room, history, connection state, live update subscription.

Frontend data modes:

? Fixture mode: `VITE_USE_FIXTURES=true`. Uses `apps/web/src/api/fixtures.ts`.

? API mode: `VITE_USE_FIXTURES=false`. Calls backend APIs.

Runtime validation:

? `apps/web/src/api/client.ts` validates dashboard responses with Zod.

Live updates:

? Uses `EventSource` against `/api/live`.

? Falls back to polling if SSE fails.

? Fixture mode simulates live changes and polling.

7. Admin Panel

Main file: `apps/web/src/admin/AdminApp.tsx`

Admin routes:

? `/admin/login`

? `/admin`

? `/admin/users`

? `/admin/ip-allowlist`

? `/admin/thresholds`

? `/admin/reports`

? `/admin/alerts`

? `/admin/system-health`

? `/admin/audit-logs`

? `/admin/settings`

Admin features:

? Secure login form.

? Current-user session check.

? Sidebar navigation.

? User management.

? IP allowlist management.

? Threshold management.

? Excel report download.

? Alerts and sensor faults.

? System health.

? Audit logs.

? Live data settings.

? Toast messages.

? Loading, error, and empty states.

Admin API client:

- ? apps/web/src/api/adminClient.ts
- ? Fetches CSRF token.
- ? Sends X-CSRF-Token for non-GET requests.
- ? Uses credentials: include for HttpOnly cookie auth.
- ? Downloads report files as blobs.

8. Backend API Contract

Primary base path: /api

Compatibility base path: /api/v1

8.1 Health And Monitoring

- ? GET /api/health: basic process health.
- ? GET /api/ready: database readiness check.
- ? GET /api/metrics: Prometheus-compatible text metrics.

8.2 Dashboard

- ? GET /api/dashboard/overview
- ? GET /api/dashboard/summary
- ? GET /api/dashboard/rooms
- ? GET /api/dashboard/rooms/{room_id}
- ? GET /api/dashboard/rooms/{room_id}/history?hours=24&limit=1000
- ? GET /api/dashboard/alerts
- ? GET /api/live

/api/live emits dashboard.snapshot Server-Sent Events.

8.3 Auth

- ? GET /api/auth/csrf
- ? POST /api/auth/login
- ? GET /api/auth/me
- ? POST /api/auth/logout
- ? GET /api/auth/bootstrap-token in non-production only.

8.4 Ingestion

- ? POST /api/ingest/records
- ? POST /api/ingest/excel

? POST /api/ingest/poll-now

Ingestion endpoints require authenticated admin/operator access. Excel and poll-now also require CSRF.

8.5 Admin

? GET /api/admin/overview

? GET /api/admin/rooms

? GET/POST/PUT/DELETE /api/admin/users

? GET/POST/PUT/DELETE /api/admin/ip-allowlist

? GET/POST/PUT/DELETE /api/admin/thresholds

? GET /api/admin/thresholds/effective

? GET /api/admin/reports/export

? GET /api/admin/alerts

? GET /api/admin/sensor-faults

? GET /api/admin/system-health

? GET /api/admin/audit-logs

? GET /api/admin/settings

? PUT /api/admin/settings/{key}

9. Database Schema

Core tables:

? roles: Role records.

? users: User records with bcrypt password hashes.

? rooms: 18 room records.

? sensors: 90 expected source sensors.

? thresholds: Global/group/room/sensor threshold overrides.

? raw_ingest_logs: Source payloads and dedupe keys.

? temperature_readings: Time-series readings.

? current_room_snapshots: Latest dashboard-ready room states.

? alerts: Warning and critical conditions.

? sensor_faults: Missing, invalid, unavailable, stale, or offline sensor states.

? ip_allowlist: CIDR allowlist entries by scope.

? report_downloads: Report export audit metadata.

? audit_logs: Admin, login, report, and block events.

? system_settings: Admin-managed settings.

? ingestion_status: Data source health.

Important indexes and constraints:

? Unique rooms.room_number.

- ? Unique sensors.source_tag.
- ? Unique raw_ingest_logs.source, source_timestamp.
- ? Unique temperature_readings.raw_ingest_log_id, source_tag.
- ? Index temperature_readings.logged_at.
- ? Index temperature_readings.room_id, logged_at.

10. Authentication And Permission Flow

10.1 Login Flow

```
Browser loads /admin/login
-> GET /api/auth/csrf
-> CSRF cookie returned
-> POST /api/auth/login with email/password and X-CSRF-Token
-> backend verifies bcrypt password hash
-> backend writes login audit log
-> backend sets HttpOnly session cookie
-> frontend calls /api/auth/me
-> admin UI opens
```

10.2 Session Handling

- ? Session token is a JWT stored in an HttpOnly cookie named cold_room_session.
- ? Cookie max age is 1800 seconds.
- ? API_COOKIE_SECURE=true should be enabled in production.
- ? CSRF cookie is named cold_room_csrf.
- ? Non-GET state changes require X-CSRF-Token.

10.3 Roles

Seeded roles:

- ? admin: intended full administrative access.
- ? operator: intended operational administration.
- ? viewer: intended read-only dashboard access.

Current backend behavior:

- ? The dependency named admin_user currently accepts both admin and operator.
- ? This means operator accounts can access sensitive admin mutation endpoints such as user deletion, threshold changes, allowlist edits, settings changes, and ingest actions.
- ? This is a delivery risk. Split permissions should be implemented before strict client handover.

Recommended role model:

Action	Admin	Operator	Viewer
View dashboard	yes	yes	yes
View admin overview	yes	yes	no
Manage users	yes	no	no

Action	Admin	Operator	Viewer
Manage IP allowlist	yes	no	no
Manage thresholds	yes	optional	no
Download reports	yes	yes	no
Manage system settings	yes	no	no
Ingest/poll data	yes	optional	no
View audit logs	yes	no or read-only	no

11. IP Allowlist Flow

Scopes:

? dashboard_access: dashboard and live update APIs.

? admin_access: auth and admin APIs.

? api_access: general API fallback scope.

Behavior:

```
Request enters backend middleware
-> determine route scope from path
-> get client IP
-> if trusted proxy mode is disabled, use socket client IP
-> if trusted proxy mode is enabled, use X-Forwarded-For
-> load active DB allowlist entries for route scope or api_access
-> combine with environment API_IP_ALLOWLIST
-> match IP against CIDRs
-> allow request or return 403
-> write blocked_access audit log when DB is available
```

Important security point:

? Do not enable API_TRUSTED_PROXY_ENABLED=true unless API_TRUSTED_PROXY_CIDRS is configured for infrastructure controlled by the client.

? Do not trust user-supplied X-Forwarded-For directly from the internet.

12. Threshold And Status Flow

Threshold priority:

1. Sensor rule.
2. Room rule.
3. Group rule.
4. Global rule.
5. Development default threshold fallback.

Status outputs:

```
? normal
? warning
? critical
```

? fault

? offline

Quality outputs:

? good

? missing

? invalid

? unavailable

? impossible

? stale

Threshold changes affect newly ingested data and recalculated snapshots. Existing historical readings are not automatically rewritten unless a backfill/reprocess job is implemented.

13. Excel Report Flow

Frontend:

```
/admin/reports
-> choose duration or custom date range
-> choose rooms/groups
-> include/exclude alerts
-> include/exclude faults
-> click Download Excel
```

Backend:

```
GET /api/admin/reports/export
-> authenticate current user
-> validate duration/date range
-> filter rooms and groups
-> count matching temperature readings
-> reject export if count exceeds API_REPORT_MAX_ROWS
-> build workbook with openpyxl
-> add report_downloads row
-> write audit log
-> return .xlsx stream
```

Workbook sheets:

1. Summary
2. Room Latest Snapshot
3. Temperature Readings
4. Alerts
5. Sensor Faults
6. Metadata

Range controls:

? Supported durations: last 1 hour, 8 hours, 24 hours, 7 days, 30 days.

? Custom ranges cannot exceed 31 days.

? API_REPORT_MAX_ROWS limits large exports.

14. Live Data Flow

There are two data providers:

? `excel_fixture`: reads rows from a configured Excel file path.

? `http_live_api`: polls an external HTTP API.

Provider selection:

```
API_DATA_PROVIDER=excel_fixture
```

or

```
API_DATA_PROVIDER=http_live_api
API_LIVE_SOURCE_URL=https://provider.example/api/temperature
API_LIVE_SOURCE_TOKEN=<secret-token>
API_POLL_INTERVAL_SECONDS=60
```

Runtime flow:

```
FastAPI startup
-> if API_DATA_PROVIDER=http_live_api, start polling loop
-> provider fetches latest rows
-> ingestion service normalizes and persists rows
-> dashboard snapshots update
-> /api/live sends snapshot events
-> frontend updates without full page refresh
-> if SSE fails, frontend falls back to polling
```

The live provider accepts:

? a single JSON row,

? a list of rows,

? or an object with a data array.

15. Deployment

15.1 Local Without Docker

Use this when no SQL Server URL is available:

```
Copy-Item .env.example .env
# Edit .env and set DATABASE_URL=sqlite+aiosqlite:///./local.db
powershell -ExecutionPolicy Bypass -File scripts/dev.ps1
```

Open:

? Dashboard: `http://localhost:5173`

? Admin: `http://localhost:5173/admin/login`

? API health: `http://localhost:8000/api/health`

? API readiness: `http://localhost:8000/api/ready`

Stop:

```
powershell -ExecutionPolicy Bypass -File scripts/stop-dev.ps1
```

15.2 Local With Docker Compose

```
Copy-Item .env.example .env
# Replace secrets and SQL password in .env
docker compose -f infra/docker-compose.yml --env-file .env up --build
```

Local Docker services:

- ? SQL Server Developer container on internal network.
- ? FastAPI API on port 8000.
- ? Vite web dev server on port 5173.
- ? Nginx local edge proxy on port 8080.

15.3 Production Docker Compose

```
cp .env.example .env
# Replace all secrets, domains, and DB settings.
docker compose -f infra/docker-compose.prod.yml --env-file .env up -d --build
```

Production services:

- ? SQL Server container on private internal network.
- ? One-shot migrate service.
- ? One-shot seed service.
- ? FastAPI API container.
- ? Static React web container served by Nginx.
- ? Nginx HTTPS reverse proxy.

15.4 HTTPS

Production Nginx expects:

```
infra/certs/fullchain.pem
infra/certs/privkey.pem
```

Recommended production settings:

```
ENVIRONMENT=production
API_COOKIE_SECURE=true
API_ALLOWED_ORIGINS=https://your-domain.example
VITE_API_BASE_URL=/api
VITE_USE_FIXTURES=false
```

16. Environment Variables

16.1 Required Backend Variables

Variable	Purpose
ENVIRONMENT	development, test, or production.
APP_NAME	API application name.
API_HOST	Bind host.

Variable	Purpose
API_PORT	Bind port.
API_SECRET_KEY	JWT signing secret. Must be strong and not placeholder.
API_ALLOWED_ORIGINS	CORS origin list.
API_COOKIE_SECURE	Enables secure cookies behind HTTPS.
API_RATE_LIMIT_PER_MINUTE	General in-memory rate limit.
API_LOGIN_RATE_LIMIT_PER_MINUTE	Login rate limit.
API_DATA_PROVIDER	excel_fixture or http_live_api.
API_POLL_INTERVAL_SECONDS	Live API polling interval.
API_REPORT_MAX_ROWS	Maximum rows allowed in Excel export.
DATABASE_URL	SQLAlchemy database URL.

16.2 Optional Backend Variables

Variable	Purpose
API_TRUSTED_PROXY_ENABLED	Allows trusted proxy IP extraction.
API_TRUSTED_PROXY_CIDRS	Intended trusted proxy CIDRs.
API_IP_ALLOWLIST	Environment-level CIDR allowlist.
API_FIXTURE_EXCEL_PATH	Excel fixture path.
API_LIVE_SOURCE_URL	Live API URL.
API_LIVE_SOURCE_TOKEN	Live API token.
API_OPENAPI_ENABLED_IN_PRODUCTION	Allows OpenAPI docs in production if true.
API_BOOTSTRAP_ADMIN_EMAIL	Optional first admin email.
API_BOOTSTRAP_ADMIN_PASSWORD	Optional first admin password.

16.3 SQL Server Variables

Variable	Purpose
MSSQL_DATABASE	Database name.
MSSQL_SA_PASSWORD	SQL Server SA password for container bootstrapping.

16.4 Frontend Variables

Variable	Purpose
VITE_API_BASE_URL	API base URL.
VITE_USE_FIXTURES	Enables/disables frontend fixture data.

17. Security Controls

Implemented controls:

- ? Proprietary license.
- ? No real secrets in tracked source files found during audit.
- ? .env is ignored by Git.
- ? .env.example is tracked as a safe template.

- ? Passwords stored as bcrypt hashes.
- ? HttpOnly session cookie.
- ? CSRF token for state-changing requests.
- ? Login rate limiting.
- ? General API rate limiting.
- ? CORS controlled by environment.
- ? IP allowlist middleware.
- ? Backend does not trust X-Forwarded-For unless trusted proxy mode is enabled.
- ? Security headers in API and Nginx.
- ? Audit logs for login success/failure, logout, admin changes, blocked access, and report downloads.
- ? SQL Server is private in Compose networking.
- ? Production OpenAPI disabled by default.

Security concerns to address:

- ? Operator role currently has broad admin API access.
- ? Rate limiting is in-memory and not suitable for multi-instance production.
- ? JWT session invalidation is not server-side; logout deletes cookie, but issued tokens remain valid until expiry.
- ? API_TRUSTED_PROXY_CIDRS is configured but not currently enforced in the `client_ip` helper; trusted proxy mode should validate the socket IP belongs to a trusted proxy before using X-Forwarded-For.
- ? Some admin forms are functional but basic; add stricter field-level validation and confirmation dialogs for every destructive action.
- ? Report download is a GET request with cookie auth. This is common for downloads, but sensitive exports should also consider short-lived signed report IDs or POST-to-create/report job patterns.

18. Testing And QA Status

Commands run during this audit on 2026-06-04:

Area	Command	Result
Backend tests	<code>PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 python -m pytest tests</code>	13 passed, 10 warnings
Frontend typecheck/lint	<code>npm run lint</code>	passed
Frontend unit tests	<code>npm test</code>	2 files, 5 tests passed
Frontend production build	<code>npm run build</code>	passed
Playwright e2e	<code>npm run e2e -- --project=chromium</code>	2 passed
Docker local check	<code>docker --version</code>	failed: Docker not installed

Known warnings:

- ? `python-jose` dependency warning about deprecated `datetime.utcnow()` use.
- ? `openpyxl` warning about workbook default style in the sample workbook.
- ? Frontend build output includes a large JS bundle around 660 KB before gzip; code splitting should be considered.

19. CI/CD

GitHub Actions workflow: `.github/workflows/ci.yml`

CI jobs:

? Backend:

? install Python dependencies,

? run Ruff check,

? run Alembic migration,

? run seed command,

? run pytest,

? run pip-audit as non-blocking advisory step.

? Frontend:

? install npm dependencies,

? run typecheck/lint,

? run tests,

? run build,

? install Playwright Chromium,

? run e2e.

? Docker:

? build backend production Dockerfile,

? build frontend production Dockerfile.

20. Troubleshooting

20.1 Backend says No module named app

Run Uvicorn from `services/api`:

```
cd services/api
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Or use:

```
powershell -ExecutionPolicy Bypass -File scripts/dev.ps1
```

20.2 Admin login does not work

Check:

? Backend is running.

? `/api/health` returns `{"status": "ok"}`.

? `/api/ready` returns `{"status": "ready"}`.

? `.env` has bootstrap admin email/password.

? `python -m app.scripts.seed` has been run.

- ? Frontend points to the correct backend with VITE_API_BASE_URL.
- ? Browser network tab shows successful CSRF request before login.

20.3 403 Forbidden on dashboard or admin

Possible causes:

- ? IP allowlist blocks current IP.
- ? Trusted proxy mode is enabled incorrectly.
- ? Request is coming through a proxy but backend sees proxy IP.
- ? Admin scope and dashboard scope are configured differently.

20.4 SQL Server connection fails

Check:

- ? SQL Server is running.
- ? Database exists.
- ? ODBC Driver 18 is installed for non-Docker API host.
- ? DATABASE_URL driver name is correct.
- ? API host can reach SQL Server network.
- ? SQL credentials are correct.

20.5 Report export fails

Check:

- ? User is authenticated.
- ? Date range is valid.
- ? Range is 31 days or less.
- ? Matching row count is below API_REPORT_MAX_ROWS.
- ? Database has readings for selected rooms/groups.

20.6 Docker commands fail

Install and start Docker Desktop or run deployment on a server with Docker Engine and Docker Compose plugin.

21. Maintenance Guide

Daily:

- ? Check /api/ready.
- ? Review admin system health.
- ? Check ingestion last success time.
- ? Review open alerts and sensor faults.

Weekly:

- ? Review audit logs.
- ? Review failed login attempts and blocked access attempts.
- ? Confirm IP allowlist entries are still valid.
- ? Confirm report export works.

Monthly:

- ? Test database backup restore.
- ? Rotate live API token if policy requires.
- ? Review admin users and remove unused accounts.
- ? Review dependency advisories.
- ? Check disk usage for SQL Server volume and backups.

Before every release:

- ? Run backend tests.
- ? Run frontend lint, unit tests, build, and e2e.
- ? Run Alembic migration check.
- ? Build Docker images.
- ? Review `.env.example` changes.
- ? Review security checklist.

22. Backup And Restore

SQL Server container backup example:

```
docker compose -f infra/docker-compose.prod.yml --env-file .env exec db /opt/mssql-tools18/bin/
sqlcmd \
-S localhost -U sa -P "$MSSQL_SA_PASSWORD" -C \
-Q "BACKUP DATABASE [$MSSQL_DATABASE] TO DISK = N'/backups/cold_room.bak' WITH INIT"
```

Restore should be tested in a staging environment before client production usage.

Recommended policy:

- ? Daily full backups.
- ? Transaction log backups every 15-60 minutes if full recovery is enabled.
- ? 30-day retention.
- ? Encrypted off-host copy.
- ? Monthly restore drill.

23. Audit Findings

23.1 Positive Findings

- ? Repository is initialized and pushed to GitHub.
- ? .env is ignored and was not tracked.
- ? Local SQLite DB files are ignored and were not tracked.
- ? Logs and node_modules are ignored.
- ? Proprietary license is present.
- ? Backend tests pass.
- ? Frontend typecheck, unit tests, build, and e2e pass.
- ? SQL Server is the documented production database.
- ? Docker Compose does not publicly expose the database.
- ? Report export is protected by authentication and audited.
- ? Missing sensor data is handled and tested.
- ? IP allowlist behavior is tested.
- ? Threshold priority behavior is tested.
- ? Admin login/auth flow is tested.

23.2 Missing Or Incomplete Items

- ? Docker is not installed on the audit workstation, so Docker image builds were not locally verified.
- ? No real live API credentials or exact provider contract are available.
- ? No production TLS certificate files are present, which is expected because certificates should not be committed.
- ? No production secret manager integration is implemented; secrets are environment-based.
- ? No distributed rate limiter such as Redis is implemented.
- ? No background worker queue for very large report generation.
- ? No server-side session revocation table.
- ? No granular role/permission matrix enforcement beyond broad admin/operator access.
- ? No automated Lighthouse report is checked into the repo.
- ? No production monitoring stack files for Prometheus/Grafana are included; /api/metrics and docs are present.

23.3 Security Gaps And Risks

High priority:

- ? Operator users can currently access sensitive admin endpoints because admin_user allows admin and operator.
- ? Trusted proxy CIDR settings are documented but not enforced in client_ip; enabling trusted proxy mode without socket-IP validation could allow spoofed X-Forwarded-For.
- ? In-memory rate limiting is not enough for multi-container or multi-server production.

Medium priority:

- ? JWT sessions are stateless and cannot be revoked before expiry except by rotating API_SECRET_KEY.
- ? GET report download with cookie auth can be improved with report jobs or one-time signed download URLs.
- ? Admin settings page stores arbitrary JSON-like values; stricter key allowlisting is recommended for production.
- ? Destructive admin actions should require confirm dialogs everywhere.

? SQL Server Compose uses SA bootstrap credentials; production should use a least-privilege runtime DB user.

Low priority:

? Build uses latest npm packages in several dependencies; pinning versions can improve repeatability.

? Large frontend bundle should be split by admin/dashboard routes.

? Some docs and UI copy still use development/demo language and should be client-branded before handover.

23.4 Hardcoded Secret Review

Tracked source scan found no real production secrets.

Allowed/test-only values found:

? `.env.example` contains placeholder values.

? Backend tests contain test-only admin email/password.

? Documentation contains example URLs and placeholder tokens.

Important:

? Local `.env` exists but is ignored by Git. It should never be shared or committed.

? Any bootstrap admin password used during development must be rotated before production.

23.5 Broken Code Review

No broken code was detected by the executed backend tests, frontend typecheck, unit tests, production build, or Playwright e2e.

Operational blockers:

? Docker is unavailable on this workstation.

? Docker image build verification must run on CI or another machine.

24. Recommended Improvements Before Client Delivery

1. Split admin, operator, and viewer permissions in backend dependencies.
2. Enforce `API_TRUSTED_PROXY_CIDRS` before trusting `X-Forwarded-For`.
3. Add Redis or gateway-based distributed rate limiting.
4. Add server-side session revocation or short-session plus refresh-token rotation.
5. Add route-level code splitting in frontend.
6. Add Prometheus/Grafana deployment examples.
7. Add production DB least-privilege user setup docs/scripts.
8. Add background report generation for large date ranges.
9. Add stricter admin form validation.
10. Add full admin e2e tests against a running backend.
11. Run Docker builds and compose smoke test on a Docker-enabled machine.

12. Validate real live API integration once credentials and provider schema are supplied.
13. Run dependency audit in CI and triage advisory output before go-live.
14. Create client-specific operations runbook with real domain, IP ranges, and backup location.

25. Client Delivery Checklist

Before final client handover:

- ? Confirm repository is private.
- ? Confirm proprietary license is present.
- ? Confirm no real secrets are committed.
- ? Confirm production `.env` is stored securely.
- ? Confirm first admin account is created.
- ? Rotate bootstrap password.
- ? Configure dashboard/admin IP allowlist.
- ? Configure SQL Server production database.
- ? Configure HTTPS certificate.
- ? Configure live API URL and token.
- ? Run migrations.
- ? Run seed command.
- ? Run full QA suite.
- ? Run Docker image build.
- ? Verify `/api/health`, `/api/ready`, and `/api/metrics`.
- ? Verify dashboard shows 18 rooms.
- ? Verify missing sensors show as faults/unavailable.
- ? Verify admin login.
- ? Verify report export.
- ? Verify audit logs.
- ? Verify backup and restore.

26. Quick Command Reference

Local dev:

```
powershell -ExecutionPolicy Bypass -File scripts/dev.ps1
```

Stop local dev:

```
powershell -ExecutionPolicy Bypass -File scripts/stop-dev.ps1
```

Backend:

```
cd services/api
python -m alembic upgrade head
python -m app.scripts.seed
python -m pytest tests
```

```
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Frontend:

```
cd apps/web
npm install
npm run lint
npm test
npm run build
npm run e2e
```

Production:

```
docker compose -f infra/docker-compose.prod.yml --env-file .env up -d --build
docker compose -f infra/docker-compose.prod.yml --env-file .env run --rm migrate
docker compose -f infra/docker-compose.prod.yml --env-file .env run --rm seed
```

27. Conclusion

The repository is a strong production-ready scaffold and working initial implementation for the Cold Room Temperature Monitoring system. The dashboard, backend normalization, admin panel, SQL schema, reporting, security middleware, deployment files, tests, and documentation are present.

The main items to resolve before strict client delivery are permission hardening, trusted proxy CIDR enforcement, distributed rate limiting, Docker build verification, and live API validation with real credentials.